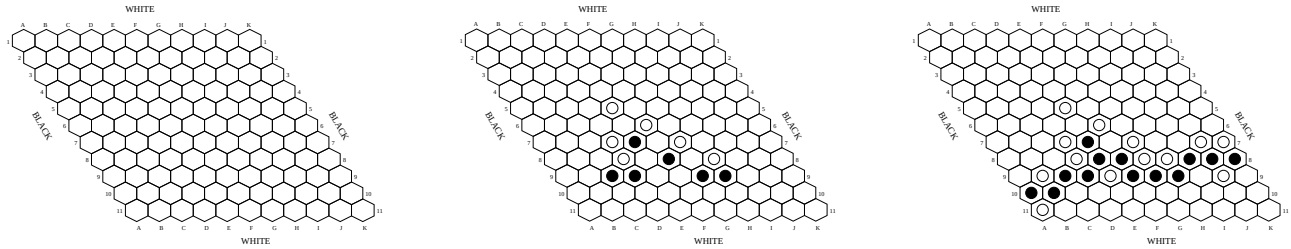
INTRODUCTION TO PROGRAMMING (2018-19)

Assignment 4

In this assignment, you are asked to implement a C program that will play, against a human opponent (the user of the program), the game *Hex*.

Hex is played on a board of hexagons, with dimension usually 11×11, but it can also be played in any dimension $N \times N$, with $N \geq 4$. One player is white and the other is black. Each player has at his disposal a sufficient number of circular tiles of his color. Also, each pair of opposite sides of the board belongs to each of the two players. In the game, the players play alternately, with white playing first, placing a tile of their color on any free position of the board they wish. Their goal is to form a continuous sequence of tiles of their color, on neighboring hexagons, that connects the two sides of the board that belong to them. Whichever player achieves it first, is the winner.

In figure (a) below the board of the game is shown before its development begins. In (b) the state of the game is shown, after the players have alternately placed some tiles. In (c) the black player has won, because he has constructed a continuous sequence of neighboring black tiles that connects the sides that belong to him.



(a) (b) (c)

Because in this game the player who plays first has an advantage, sometimes, it is agreed between the two players to also apply a rule called the *swap rule*. According to this rule, after the first move of the white player, black has the right, if he wishes, to become the first player himself with first move the symmetric of the one that the white player played. By the term "symmetric", we mean the interchange of column and row with respect to the move of white. For example, if white had played his first move at D2, black has the right to cancel it, when the swap rule is in effect, and to play himself as first move at B4. Then, the white player plays, as second.

Hex is a strategy game and developing algorithms so that a computer can play this game well is the subject of a branch of Computer Science, which is called *Artificial Intelligence*. You can find more information about the game at the link `http://en.wikipedia.org/wiki/Hex_(board_game)`.

The program that you will write (let us assume that the executable is called "`hex`") will be able to accept from the command line some options, for setting initial parameters, and after it starts executing, it must accept commands from a predefined repertoire that will determine its operation.

The options from the command line are the following:

`-n <size>` : Standard Hex is played on an 11×11 board. Your program must be able to play the game for any size of the board dimension (`<size>`) between 4 and 26 (inclusive), which is specified by this option. If the option is not given, it should be assumed that `<size> = 11`.

`-d <difficulty>` : Sets the difficulty level at which the game will be played by the program that you will write. If the option is not given, it should be assumed `<difficulty> = 1`, which is

the lowest difficulty level. More about the difficulty level, below.

`-b` : The user will be the black player, so he plays second. If the option is not given, the user will be the white player, so he plays first.

`-s` : The swap rule is active. If the option is not given, the swap rule is not applied.

After the program that you will write starts executing, it should accept commands that determine parameters, but also the operation of the game. These commands are:

`newgame [white|black [swapoff|swapon [<size>]]]` : Starts a new game from the beginning. If no parameter is given, the color of the user, whether the swap rule is in effect and the size of the game result from what was given on the command line. If one parameter is given, it must be one of `white` or `black`, which is the color that the user will have. If, besides the first, a second parameter is also given, it must be one of `swapoff` or `swapon`. In the first case, the swap rule is not applied, while in the second it is applied. If, besides the first two, a third parameter is also given, it is the size of the game.

`play <move>` : The command is acceptable when it is the user's turn to play. The `<move>` is the move that the player wishes to make, in the form `<letter><number>`, the coordinates of the hexagon where he wishes to place a tile of his color, where `<letter>` is one of (`A`, `B`, `C`, ...) and `<number>` is one of (`1`, `2`, `3`, ...).

`cont` : The command is acceptable when it is the program's turn to play, in which case the move that it decides is made.

`undo` : The user undoes his previous move. If the move of the program has also already been made, it is undone as well. Consecutive `undo` commands are also allowed.

`suggest` : The user asks the program to suggest to him the move that would be good for him to make. The command is acceptable when it is his turn to play.

`level [<difficulty>]` : The user selects as difficulty level the `<difficulty>`. If `<difficulty>` is not given, the program prints which is the current difficulty level of the game.

`swap` : If the user plays second, with this command he can indicate, only on his first move, and provided that the swap rule is in effect, that he wishes a "swap" to be made.

`save <statefile>` : Saves the current state of the game to the file `<statefile>`.

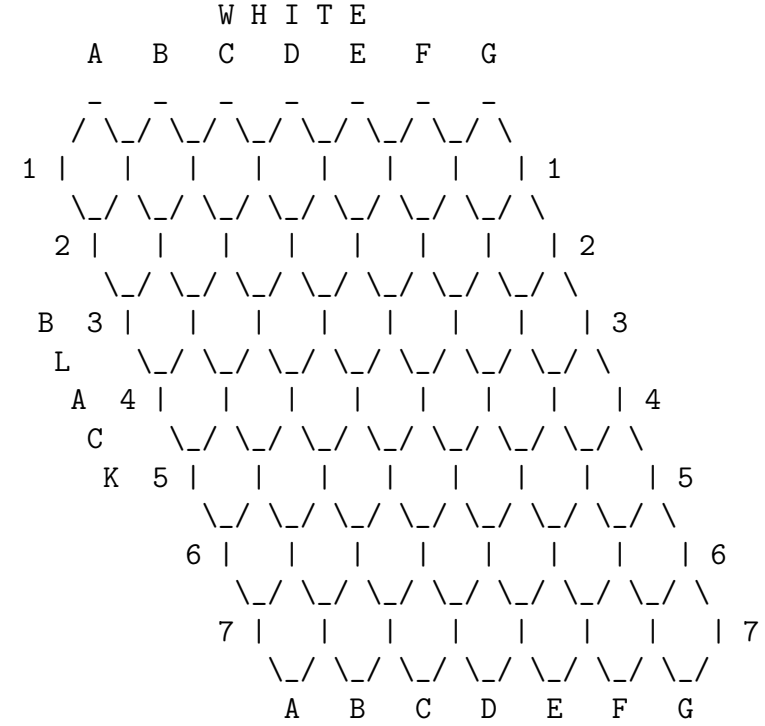
`load <statefile>` : Loads from the file `<statefile>` a saved game state. More about the format of the state files, below.

`showstate` : The current state of the game is printed.

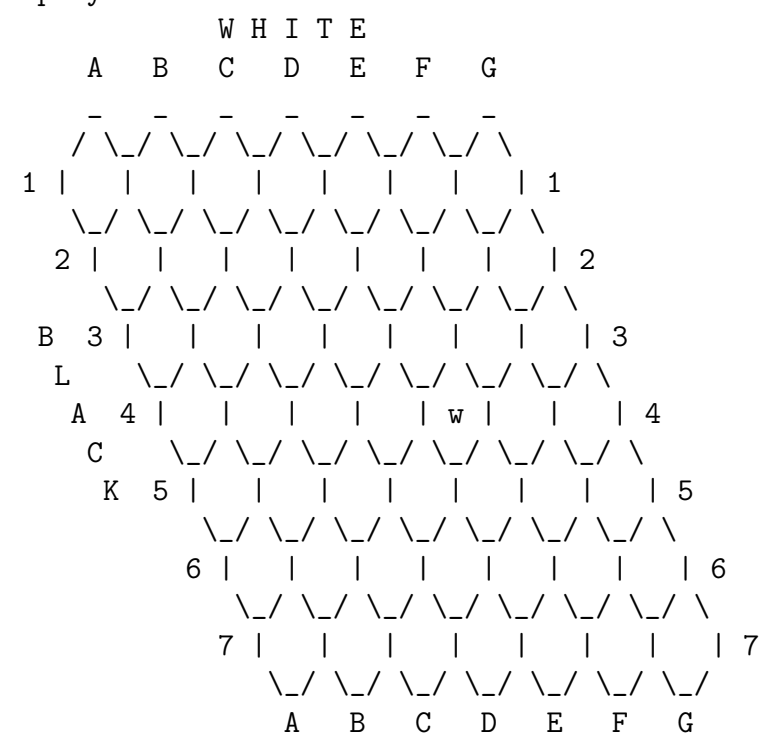
`quit` : Terminates the program.

An indicative execution of the program is shown below:

```
$ ./hex -n 7 -d 2
> newgame
```




```
White player (human) plays now
> play E4
```



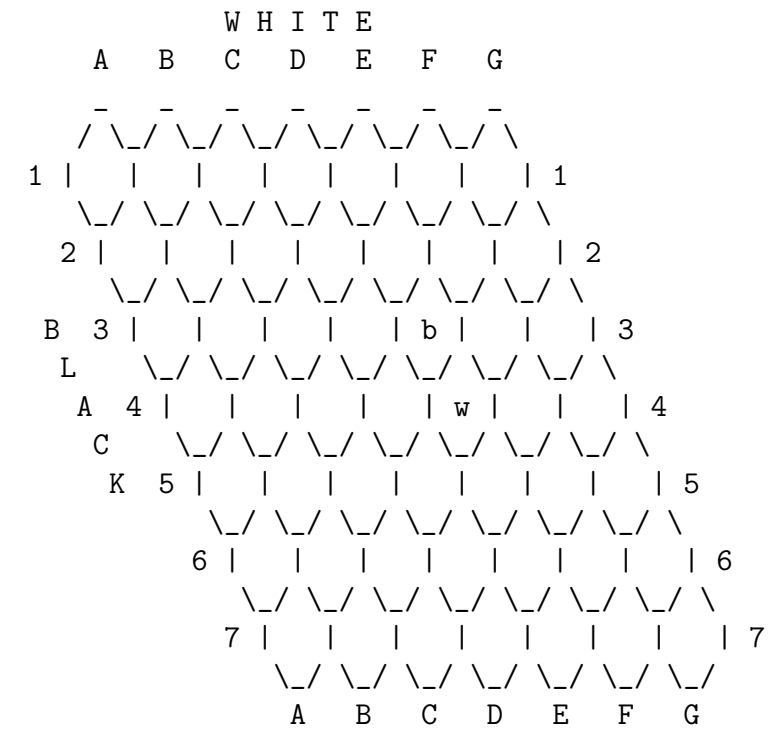

```
Move played: E4
Black player (computer) plays now
```

```
> cont
             W H I T E
     A   B   C   D   E   F   G
     _   _   _   _   _   _   _
    / \_/ \_/ \_/ \_/ \_/ \_/ \
 1 |   |   |   |   |   |   |   | 1
    \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
   2 |   |   |   |   |   |   |   | 2
      \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
  B  3 |   |   |   |   | b |   |   | 3
   L    \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
    A  4 |   |   |   |   | w |   |   | 4
     C    \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
      K  5 |   |   |   |   |   |   |   | 5
            \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
           6 |   |   |   |   |   |   |   | 6
              \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
             7 |   |   |   |   |   |   |   | 7
                \_/ \_/ \_/ \_/ \_/ \_/ \_/
                  A   B   C   D   E   F   G

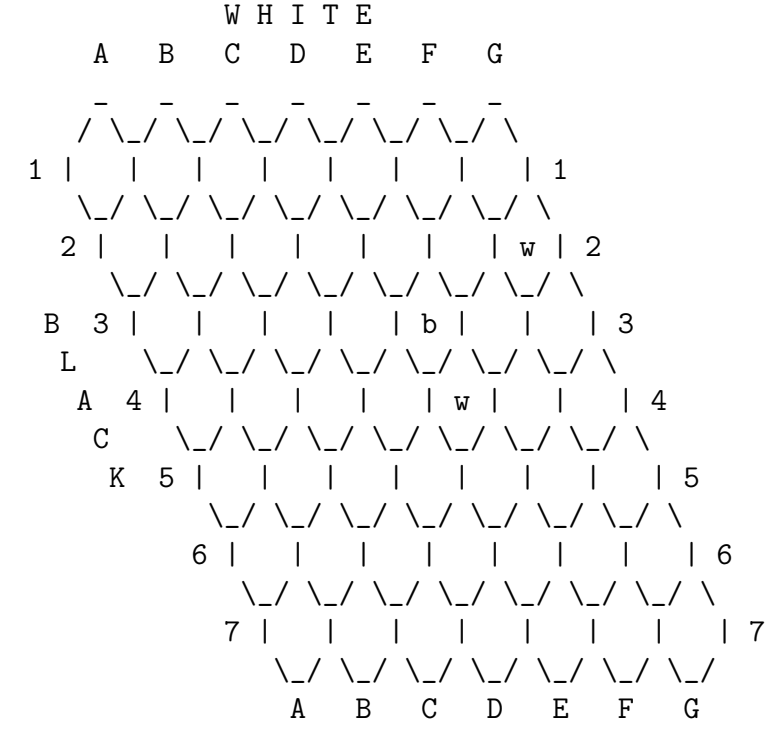
Move played: E3
White player (human) plays now
> play D3
             W H I T E
     A   B   C   D   E   F   G
     _   _   _   _   _   _   _
    / \_/ \_/ \_/ \_/ \_/ \_/ \
 1 |   |   |   |   |   |   |   | 1
    \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
   2 |   |   |   |   |   |   |   | 2
      \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
  B  3 |   |   |   | w | b |   |   | 3
   L    \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
    A  4 |   |   |   |   | w |   |   | 4
     C    \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
      K  5 |   |   |   |   |   |   |   | 5
            \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
           6 |   |   |   |   |   |   |   | 6
              \_/ \_/ \_/ \_/ \_/ \_/ \_/ \
             7 |   |   |   |   |   |   |   | 7
                \_/ \_/ \_/ \_/ \_/ \_/ \_/
                  A   B   C   D   E   F   G

Move played: D3
Black player (computer) plays now
```

```
> undo
```




```
White player (human) plays now
> play G2
```




```
Move played: G2
Black player (computer) plays now
> save file.hex
> quit
```

```
$ ./hex
> load file.hex
> ................
```

Another indicative execution of the program, for a game on a 4×4 board, is the following:

```
$ ./hex
> newgame black swapon 4
       W H I T E
     A   B   C   D
     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   |   |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   |   |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 |   |   |   |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

White player (computer) plays now
> level 3
> cont
       W H I T E
     A   B   C   D
     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   |   |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   |   |   | w | 3
    K   \_/ \_/ \_/ \_/ \
       4 |   |   |   |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: D3
Black player (human) plays now
```

```
> swap
       W H I T E
     A   B   C   D

     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   |   |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   |   |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 |   |   | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: swap
White player (computer) plays now
> cont
       W H I T E
     A   B   C   D

     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   |   |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 |   |   | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: B3
Black player (human) plays now
> play B4
       W H I T E
     A   B   C   D

     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   |   |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 |   | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D
```

```
Move played: B4
White player (computer) plays now
> cont
       W H I T E
     A   B   C   D
     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   |   |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: A4
Black player (human) plays now
> play C1
       W H I T E
     A   B   C   D
     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 |   |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: C1
White player (computer) plays now
```

```
> cont
       W H I T E
     A   B   C   D

     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 | w |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: A2
Black player (human) plays now
> suggest
You may play at B1
> showstate
       W H I T E
     A   B   C   D

     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   |   | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 | w |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Black player (human) plays now
> play B1
       W H I T E
     A   B   C   D

     _   _   _   _
    / \_/ \_/ \_/ \
 1 |   | b | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 | w |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D
```

```
Move played: B1
White player (computer) plays now
> cont
       W H I T E
     A   B   C   D
     _   _   _   _
    / \_/ \_/ \_/ \
 1 | w | b | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 | w |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 |   | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: A1
Black player (human) plays now
> play A3
       W H I T E
     A   B   C   D
     _   _   _   _
    / \_/ \_/ \_/ \
 1 | w | b | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 | w |   |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 | b | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: A3
White player (computer) plays now
```

```
> cont
       W H I T E
     A   B   C   D
      _   _   _   _
    / \_/ \_/ \_/ \
 1 | w | b | b |   | 1
B   \_/ \_/ \_/ \_/ \
 L 2 | w | w |   |   | 2
  A   \_/ \_/ \_/ \_/ \
   C 3 | b | w |   |   | 3
    K   \_/ \_/ \_/ \_/ \
       4 | w | b | b |   | 4
          \_/ \_/ \_/ \_/
           A   B   C   D

Move played: B2

End of game
White player (computer) wins with path A1-A2-B2-B3-A4
> quit
```

As for the format of the state files, which are handled by the commands `save` and `load`, it must be the following: Initially, inside the file there is (in binary form) an integer, of one byte, which is the dimension of the board (say n), then an indication, in one byte, of which player has the turn to play (`'w'` or `'b'`, accordingly) and then n^2 bytes, which encode the contents of the board positions, by rows. For the positions with a white tile there is the `'w'`, for those with a black tile the `'b'` and for the empty positions the `'n'` (ASCII codes). The file in which the state of the game was saved, during the first of the previous indicative executions, is `http://www.di.uoa.gr/~ip/hwfiles/hex/file.hex`.

Difficulty level - The minimax algorithm

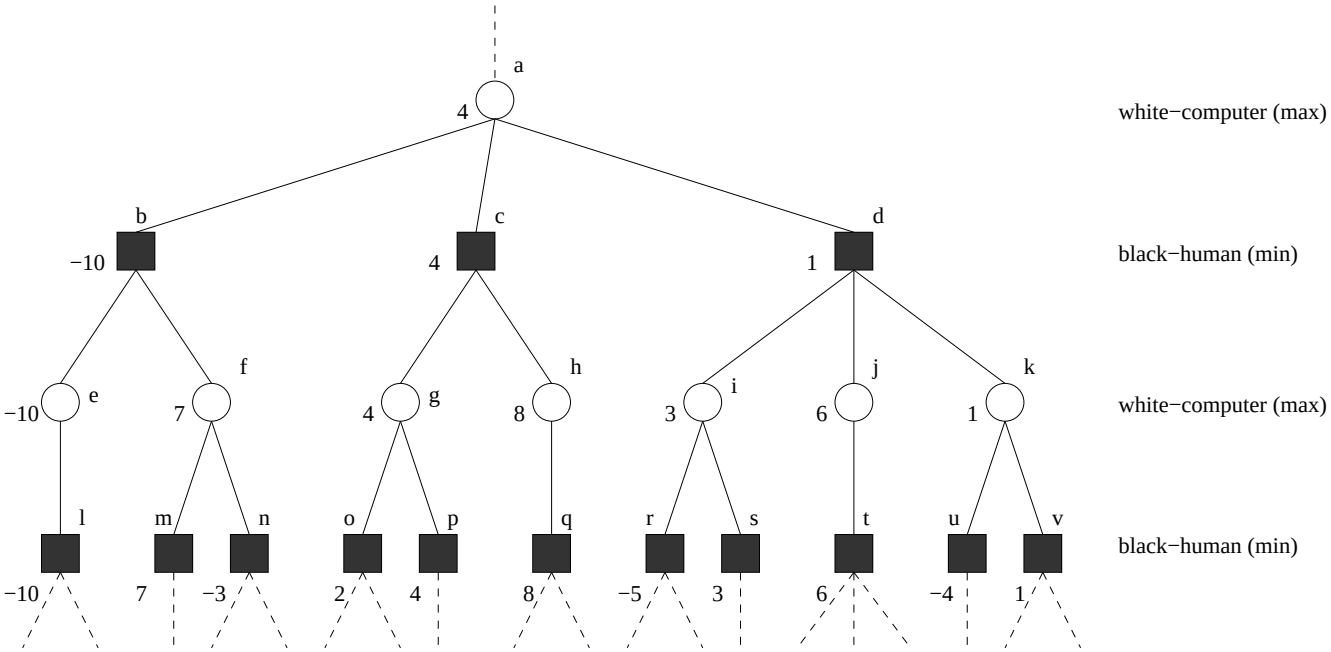
One issue that should be discussed here to some extent is that of the difficulty level at which the program that you will write will be called to play.

Suppose that the program is at the point where it must decide what move to make. But how will it choose in which free position of the board to place a tile of its color? Obviously it must be the one that will lead it to the most favorable possible state for its color. So a measure is needed of how good a game state is for a player of a given color. A very simple such measure for Hex is the difference of the length of the longest sequence of consecutive tiles of the player's color minus the corresponding sequence length for the opponent. It is part of the exercise for you to think of useful measures of the quality of a state, which you should combine appropriately so that as reliable a resultant as possible emerges.

Returning now to the problem of the selection by the program of the most suitable move among all the possible ones, a, relatively naive, approach, is to select that move which leads to a state with the best quality, for the computer-player. We could then say that the program plays at difficulty level 1. This approach is characterized as naive, because it gives some temporary advantage to the program, but it may lead to a state where the user-player has available some very crucial move, which overturns the balance of the game in his favor. Perhaps the program should foresee something like this and avoid making the wrong move? In other words, couldn't it, instead of

taking into account what may happen after one move in the current state, take into account the next two moves as well (level 2)? And why make a prediction for only two moves and not for three? You understand that this logic generalizes, so that the program can explore the possible alternatives for the next `d` moves, in which case we say that it plays at difficulty level `d`.

The method that was described above in general terms is the minimax algorithm that has been developed for two-player games, in the context of Artificial Intelligence. To better understand the logic of this algorithm, see the next figure that represents part of a *game tree*.



Suppose that at some point the game is in state a, in which it is the turn of the computer-player to play with the white tiles, and suppose that the possible moves it can make would lead the game to one of the states b, c or d. If the program were playing at difficulty level 1, it would simply evaluate the quality of these three states and would make the move that would be the most advantageous for it. Suppose however that the program has been set to play at difficulty level 3. Then it must make its decision by predicting what can happen after three moves. So, for each possible next state among b, c or d, in which it is the turn of the user-player to play with the black tiles, the program must find the next possible states, in which it will be the turn of the computer-player to play and, finally, for each one of these, find the possible next ones, in which it will be the turn of the user-player to play. These states are shown at the last level of the figure.

What the program must do is evaluate the states of the last level and propagate the evaluations upwards with the following logic. In each state where it is the turn of the computer-player to play with the white tiles, its evaluation is the maximum of the evaluations of the states that may result from it (for this reason, the computer-player is called max). The reason is that if the game finds itself in such a state and it is the turn of the computer-player to play, the program should choose the most advantageous move. Conversely, in each state where it is the turn of the user-player to play with the black tiles, its evaluation is the minimum of the evaluations of the states that may result from it (for this reason, the user-player is called min). The logic is that the user-player will make the least advantageous move for the computer-player.[1]

In the previous figure, the numbers in the nodes of the last level are the evaluations of the corresponding states, as these are computed by the function that has been selected to estimate the quality of the states from the side of the computer-player. The numbers in the intermediate

[1]From the characterizations of the two players, as max and min, also comes the name of the algorithm (minimax).

nodes are the evaluations of the corresponding states, always from the side of the computer-player, as these are propagated from the last level towards the upper ones. That is, the numbers that exist in the intermediate nodes are not the evaluation of their own quality, but result from their children. For example, i = max{r,s} = max{-5,3} = 3, k = max{u,v} = max{-4,1} = 1, so d = min{i,j,k} = min{3,6,1} = 1. Finally, for the given example, when the game state is a, the move that is advantageous for the program to make is the one that leads it to state c.

Also, for your information, let it be mentioned that there is also an algorithm for two-player games, which constitutes an improved version of minimax, the algorithm of *alpha-beta cutoffs* (alpha-beta pruning).[2] It is noted that it is not within the requirements of the exercise to implement this algorithm, but it is mentioned because it may be useful to you if you wish your assignment to participate with ambitions in the Hex cup that is described at the end of the assignment statement.

Deliverable

You must structure your program in a set of **at least three source files** C (with extension `.c`) and **at least two header files** (with extension `.h`). Also, create a simple text file named `README.txt`, in which you give instructions for the compilation of the program, as well as anything else you consider appropriate to point out. Optionally, you can also deliver a `Makefile` that takes care of the whole process of building the final executables through the command "`make`" (give "`man make`" for more details).

To deliver the set of files that you will have created for this assignment, follow the following procedure. Place all the files inside a directory that you will create on some Linux system, say with the name `hex`. Being in the directory that contains the directory `hex`, create a flattened `tar` file (say with the name `hex.tar`) that contains the directory `hex` and all its contents. This is done with the command "`tar cvf hex.tar hex`".[3] Compress the file `hex.tar`, so that the file `hex.tar.gz` is created. This is done with the command "`gzip hex.tar`".[4] The file `hex.tar.gz` is the one that you must submit through eclass.[5]

Group assignment

This assignment can also be delivered by **groups of two persons**. In this case, it will be delivered by only one member of the group, but inside the file `README.txt` the details of both members will be clearly stated. The goal of this procedure is to strengthen the idea of **equal** collaboration in a group for the achievement of a goal. If the members of the group have implemented different parts of the assignment, it must be explicitly stated in the `README` file what each member has implemented, so that in the oral examination that will follow, there is no requirement for some member of the group to have full knowledge of how the parts in which it has not been involved have been implemented.

Hex Cup (BONUS grading)

The assignments that will be submitted will be able to participate, provided that their authors declare it, in a Hex cup that will be held, with a knock out system, after the last day of the oral

[2] For more information, at `http://en.wikipedia.org/wiki/Alpha-beta_pruning`.

[3] If you want to recover the tree structure that has been stored in a flattened `tar` file `file.tar`, this can be done with the command "`tar xvf file.tar`".

[4] If you want to decompress a file `file.gz` that has been compressed with the command `gzip`, this can be done with the command "`gzip -d file.gz`".

[5] Do not submit uncompressed files or files that are compressed in a format other than `tar.gz` (e.g. `rar`, `7z`, `zip`, etc.), because they will not be accepted for evaluation.

examination of the assignments. A necessary condition for the cup to be held is that there will be at least eight entries. For the three best assignments, there will be a reward in their grade by 100%, 70% and 40%, respectively. More details about the cup will be announced in due time on the forum of the course.

The programs that will participate in the Hex cup must fulfill the following conditions:

- The program must, after it prints whatever is needed after each move (e.g. immediately before the reading of the next command), flush its output, with the command "`fflush(stdout);`".

- The program must report in one line of its output which move was made after each command `cont` but also `play`. This line must contain the two words `Move` and `played`, have the symbol of the colon and then contain the move (e.g. `Move played: D3`). The move must be a capital letter followed by one or two digits, without spaces in between (correct moves: `A2`, `F23`, wrong moves: `a2`, `F 23`, etc.). Additionally, the program can, if it plays second, choose "swap" on its first move. Then in its report, instead of position coordinates, there must be the word `swap` (that is, `Move played: swap`).

- The program must definitely implement the options `-n`, `-d`, `-b` and `-s` of the command line and the internal commands `newgame`, `play` and `cont`.

- The program must be able to respond correctly to a possible "swap" command from its opponent, regardless of whether it has implemented this capability itself.

- In case the program uses randomness for the selection of its moves, it must set the seed of the random number generator to a fixed value inside the code, so that it is possible to reproduce its results.

- Finally, the program must, at least when it plays at level 1, give the move that it decided to make in less than half a minute (real time), when it is executed on some Linux machine of the laboratory of the Department, which has no load from the execution of other programs. Additionally, if we take the sum of the real times that the program spent to make all its moves in a Hex game of dimension N, this must not exceed $N/2$ minutes.